

Análisis del repositorio mle-star-mcts-skrub

Estado actual, funcionamiento original, cambios implementados
y problemas para ejecutar con la API de la universidad

Fecha: 14 de junio de 2026

1. Resumen ejecutivo

El repositorio partió de un initial commit con solo un README.md. En el commit 338cad9 se importó el proyecto MLE-STAR de Google, una aplicación completa construida sobre el Agent Development Kit (ADK) de Google y diseñada para usar Gemini/Vertex AI. MLE-STAR es un sistema multi-agente que genera soluciones de machine learning para competiciones de Kaggle: busca modelos, genera código, lo ejecuta, lo depura, refina bloques específicos, crea ensembles y produce un submission.csv.

Los commits posteriores intentaron adaptar el proyecto para usar una API OpenAI-compatible proporcionada por la universidad (SAIA) en lugar de Gemini. Para ello se reemplazó el cliente de Google por un cliente OpenAI, se simplificó enormemente el agente principal y se añadió un ejecutor de código.

El problema central es que la adaptación desechó la arquitectura original de MLE-STAR en vez de conservarla. El código actual hace consultas exitosas a la API (sí responde el LLM), pero nunca ejecuta el código Python generado para entrenar un modelo. Por eso el usuario percibe que 'las consultas sí funcionan, pero no entrena nada'.

El camino más eficiente para lograr el objetivo actual es recuperar la arquitectura, prompts y lógica de ejecución del MLE-STAR original y reemplazar únicamente el runner de ADK por un orquestador propio que use el cliente OpenAI.

2. Cómo funciona el repositorio original (commit 338cad9)

2.1 Estructura general

El proyecto importado es una aplicación de Google ADK. Sus componentes principales son:

- machine_learning_engineering/agent.py: define root_agent, punto de entrada exigido por ADK, y mle_pipeline_agent, un SequentialAgent que coordina cuatro subagentes.
- machine_learning_engineering/sub_agents/initialization/: busca modelos candidatos, los evalúa y mezcla los mejores.
- machine_learning_engineering/sub_agents/refinement/: realiza estudios de ablación y refina bloques de código específicos.
- machine_learning_engineering/sub_agents/ensemble/: genera planes de ensemble a partir de las soluciones refinadas.
- machine_learning_engineering/sub_agents/submission/: añade código de test y produce submission.csv.
- machine_learning_engineering/shared_libraries/: contiene utilidades compartidas: code_util.py (ejecución y scoring), debug_util.py (depuración), common_util.py (extracción de texto), config.py (configuración) y check_leakage_util.py (data leakage).

2.2 El pipeline secuencial original

En agent.py original se define el pipeline así:

```
mle_pipeline_agent = agents.SequentialAgent(
    name="mle_pipeline_agent",
    sub_agents=[
        initialization_agent_module.initialization_agent,
        refinement_agent_module.refinement_agent,
        ensemble_agent_module.ensemble_agent,
        submission_agent_module.submission_agent,
    ],
)
```

Flujo:

1. Initialization: usa búsqueda web (google_search) para recuperar modelos recientes y sus códigos de ejemplo. Genera varios candidatos, los ejecuta y guarda el mejor.
2. Refinement: toma la mejor solución inicial, identifica bloques de alto impacto mediante ablaciones y los mejora iterativamente.
3. Ensemble: combina las mejores soluciones refinadas en planes de ensemble y los evalúa.
4. Submission: toma la mejor solución final, añade la carga del conjunto de test y escribe submission.csv.

Todo el estado se almacena en callback_context.state, un diccionario persistente entre agentes.

2.3 Cómo se ejecuta el código generado

La función clave es code_util.evaluate_code. Su flujo interno es: (1) recupera el código generado del estado; (2) decide el nombre del archivo Python a escribir; (3) verifica que el código sea ejecutable mediante get_run_code_condition, que exige la cadena 'Final Validation Performance' y prohíbe exit(); (4) escribe el archivo en el directorio de trabajo; (5) ejecuta el script con subprocess.run; (6) si tiene éxito, parsea la línea Final Validation Performance: <score>; (7) guarda el resultado en el estado.

La extracción del código de la respuesta del LLM ocurre en debug_util.get_code_from_response:

```
def get_code_from_response(callback_context, llm_response, do_eval=True):
    response_text = common_util.get_text_from_response(llm_response)
    code = response_text.replace("```python", "").replace("```", "")
    callback_context.state[code_state_key] = new_code
    if do_eval:
        code_util.evaluate_code(callback_context=callback_context)
```

Es decir: cada vez que un agente genera código, ese código se ejecuta inmediatamente y el resultado se guarda en el estado.

2.4 El bucle de depuración original

debug_util.get_run_and_debug_agent construye una estructura anidada: un agente run genera y ejecuta código; si falla, un run_loop reintenta; si sigue fallando, un bug_summary resume el error; un debug agent corrige el código; el bucle de depuración se repite hasta max_debug_round; el bucle completo se repite hasta max_rollback_round. Los prompts de depuración instruyen explícitamente: 'Remember to print a line in the code with Final Validation Performance: ...'

2.5 Los prompts fuerzan la ejecución

Los prompts originales (MODEL_EVAL_INSTR, CODE_INTEGRATION_INSTR, BUG_REFINE_INSTR, ENSEMBLE_PLAN_IMPLEMENT_INSTR, ADD_TEST_FINAL_INSTR) incluyen instrucciones estrictas: el código debe ser un único archivo Python autocontenido; debe usar los datos de ./input; debe imprimir Final Validation Performance: <score>; no debe usar exit(); no debe usar try/except para ocultar errores; la respuesta solo debe contener un bloque de código.

2.6 ¿Es el original un árbol de búsqueda? No, es greedy

Aunque el nombre del proyecto es MLE-STAR (Search and Targeted Refinement), el pipeline original no es un árbol de búsqueda ni MCTS. Es un pipeline greedy secuencial con loops locales de retry y debug: en initialization se generan varios candidatos, se ejecutan y se ordena por score; en refinement se prueban mejoras y se conserva la mejor; en ensemble se generan planes y se elige el mejor; en submission se compara la mejor solución refinada contra la mejor solución de ensemble. No hay backtracking, branching, rollout ni backpropagation.

3. Los cambios que has implementado

3.1 Commit 0381d7e — Docker

- Se añadió un Dockerfile basado en python:3.12-slim.
- Instala build-essential, curl, git y uv.
- Usa un truco para instalar dependencias antes de copiar el código real.
- Añade requires = ['hatchling'] a pyproject.toml.

3.2 Commit 106f630 — Configuración de la API de la universidad

- Se reescribió machine_learning_engineering/__init__.py para cargar .env, leer OPENAI_API_BASE, OPENAI_API_KEY y ROOT_AGENT_MODEL, y crear un cliente global OpenAI.
- Se eliminó toda la configuración de Google Cloud y Vertex AI.
- Se actualizó .env.example con variables del proxy SAIA.
- Se añadió prueba_api.py (script de prueba de la API).
- Se añadieron openai y python-dotenv a pyproject.toml.
- Se añadieron .gitignore y .dockerignore.
- Se borró uv.lock (3.871 líneas eliminadas).

3.3 Commit fd2f9da — Agente manager y kill switch

- Se reemplazó todo el pipeline ADK de agent.py por una clase GerenteAgent (luego renombrada a ManagerAgent).
- Usa client.chat.completions.create(...) directamente.
- Añade un contador de tokens gastados y un límite de 5.000 tokens.

3.4 Commit 8ed6658 — Protocolo de subagentes

- Se renombró GerenteAgent a ManagerAgent.
- Se añadió la clase SubAgent con un método work().
- Se añadió delegate_task() para que el manager pase el historial a un subagente y reciba su respuesta.

3.5 Commit cce0b7e — Demo multi-agente

- Se añadió un bloque __main__ que define una tarea hardcodeada sobre el dataset Titanic.
- Crea cuatro SubAgent con prompts inline.
- Los ejecuta secuencialmente: Initialization → Refinement → Ensemble → Submission.

3.6 Commit 5224689 — Ejecución de código con reintentos

- Se añadió `machine_learning_engineering/executor.py` con `extract_python_code(text)` y `run_python_script(code)`.
- Se añadió `ManagerAgent.execute_with_retries(employee, max_retries=3)`: delega, extrae código, lo ejecuta, y si falla reinyecta el error.
- Se corrigió `self.nombre` a `self.name`.
- Se añadió `skrub` como dependencia en `pyproject.toml`.

4. Problemas para que el modelo funcione con la nueva API

4.1 El problema central: se descartó la arquitectura de ejecución

La adaptación actual reemplazó el pipeline ADK por un chat multi-agente muy simplificado. Los archivos originales de `sub_agents/` y `shared_libraries/` siguen en el repositorio pero no se importan ni se usan. Como consecuencia no se copian los datos a `./input`, no se ejecuta el código generado de forma sistemática, no se parsea Final Validation Performance, no se guarda el mejor modelo, no hay bucles de depuración estructurados y no se genera `submission.csv`.

4.2 El `__main__` nunca llama a `execute_with_retries`

El bloque `__main__` de `agent.py` hace esto:

```
out_1 = Manager.delegate_task(agent_init)
out_2 = Manager.delegate_task(agent_refine)
out_3 = Manager.delegate_task(agent_ensemble)
out_4 = Manager.delegate_task(agent_submit)
```

Llama a `delegate_task`, que solo obtiene texto del LLM y lo imprime. No ejecuta código. El método que sí ejecuta código, `execute_with_retries`, no se usa en la demo.

4.3 Import roto en `agent.py`

La línea `import executor` es un `import` absoluto. Como `executor.py` está dentro del paquete `machine_learning_engineering`, Python no lo encuentra al importar `machine_learning_engineering.agent`. Debería ser:

```
from machine_learning_engineering import executor
```

Esto provoca `ModuleNotFoundError: No module named 'executor'`.

4.4 Los prompts actuales no piden código ejecutable

Los prompts inline del `__main__` piden: 'outline a clear, short 3-step plan', 'suggest 3 concrete data cleaning steps', 'suggest 2 specific Machine Learning models', 'Write a 1-paragraph executive summary'. Ninguno pide un script Python autocontenido, ni menciona `./input`, ni exige imprimir Final Validation Performance. Por tanto, incluso si se llamara a `execute_with_retries`, el LLM probablemente no devolvería código ejecutable.

4.5 El ejecutor actual es demasiado simple

- Siempre escribe en `./workspace/script.py` relativo al directorio actual.
- No copia datos de entrada.

- No parsea scores.
- No guarda el mejor código.
- Tiene un timeout de 120 segundos.
- No gestiona directorios de trabajo por tarea ni por fase.

4.6 No hay gestión de estado estructurado

El ManagerAgent solo guarda un historial de chat (self.history). No existe un objeto de estado con claves como task_description, workspace_dir, task_name, data_dir, init_code_{task_id}_{model_id}, train_code_{step}_{task_id}, ensemble_code_{iter}, best_score_{task_id} o submission_code. Sin estas claves, los subagentes no pueden construir sobre el trabajo previo de forma confiable.

4.7 El límite de tokens es irrealista

self.token_limit = 5000 es insuficiente. Una sola respuesta generando código de ML puede consumir varios miles de tokens. Con 5.000 tokens de límite total, el sistema se detendrá antes de completar el pipeline.

4.8 Tests, deployment y eval están huérfanos

- tests/test_agents.py importa root_agent.
- deployment/deploy.py importa root_agent.
- eval/simple_eval/test_eval.py y eval/full_eval/test_eval.py usan google.adk.evaluation.agent_evaluator.AgentEvaluator.
- Todos referencian el root_agent de ADK que ya no existe.

4.9 Dependencias inconsistentes

- pyproject.toml lista python-dotenv dos veces.
- Conserva dependencias pesadas de Google que el nuevo código no usa.
- Añade openai, skrub y torch.
- Se borró uv.lock sin regenerarlo, por lo que el Dockerfile ya no puede aprovechar la resolución fijada.

4.10 El archivo .env contiene credenciales reales en el workspace

Aunque .gitignore excluye .env, el archivo sigue presente en el disco del workspace con una clave de API real. Esto representa un riesgo de seguridad si se comparte el entorno.

5. Por qué 'hace consultas a la API pero no entrena nada'

Tu observación es exacta y se explica por la siguiente cadena: (1) `__main__` ejecuta cuatro `delegate_task`; (2) cada `delegate_task` llama al LLM a través de la API universitaria; (3) el LLM responde con texto (planes, sugerencias, resúmenes); (4) ese texto se añade al historial y se imprime; (5) en ningún momento se extrae código Python del texto ni se ejecuta; (6) por tanto, no se entrena ningún modelo, no se calcula ninguna métrica y no se genera `submission.csv`.

Incluso si se cambiara `delegate_task` por `execute_with_retries`, el sistema fallaría porque: los prompts no piden código ejecutable; no hay datos en `./input`; no se parsea Final Validation Performance; no se conserva la mejor solución entre fases.

6. Recomendación para ejecutar el modelo original con la API

La estrategia más eficiente es conservar la lógica de MLE-STAR original y reemplazar solo el motor de ADK. Los cambios mecánicos necesarios son los siguientes.

6.1 Mantener el cliente OpenAI (ya está hecho)

`machine_learning_engineering/__init__.py` actual ya crea el cliente correctamente:

```
from openai import OpenAI
client = OpenAI(api_key=API_KEY, base_url=API_BASE)
```

6.2 Reemplazar la extracción de texto de respuestas

En `shared_libraries/common_util.py`, cambiar `get_text_from_response` para que reciba una respuesta de OpenAI:

```
def get_text_from_response(response):
    return response.choices[0].message.content
```

6.3 Reemplazar el estado de ADK por un diccionario Python

Crear un objeto de estado simple y refactorizar `code_util.py`, `debug_util.py` y `check_leakage_util.py` para que reciban `(state, agent_name)` en lugar de `callback_context`.

6.4 Reemplazar los agentes ADK por un orquestador propio

Implementar una función `run_agent(state, agent_name, instructions, before_model, after_model, max_iter)` y reimplementar `SequentialAgent`, `ParallelAgent` y `LoopAgent` como funciones sobre este runner.

6.5 Eliminar o reemplazar google_search

- Opción A (simple): eliminar la herramienta y confiar en el conocimiento del LLM. Pierdes la 'S' de STAR pero el sistema funciona.
- Opción B (mejor): implementar un wrapper de búsqueda web que devuelva texto plano usando una API accesible.

6.6 Conservar prompts, ejecución y scoring originales

No reescribir los prompts de `sub_agents/*/prompt.py`. Ya contienen las instrucciones correctas para generar código ejecutable con Final Validation Performance. Conservar `code_util.py` para escribir archivos, ejecutar con `subprocess`, parsear el score y guardar resultados en el estado.

6.7 Conservar la gestión de workspace y datos

El original copia `tasks/<task_name>/` a `workspace/<task_name>/<stage>/input`. Esto es necesario porque el código generado asume que los datos están en `./input`.

6.8 Descartar el agent.py y executor.py actuales

El `agent.py` actual y el `executor.py` recién añadidos son demasiado simplificados y contienen errores. Es preferible reconstruir `agent.py` a partir del original, usando el nuevo orquestador y el cliente OpenAI.

6.9 Corregir dependencias

- Eliminar duplicados en `pyproject.toml`.
- Decidir si se mantienen las dependencias de Google (innecesarias para el nuevo camino).
- Regenerar `uv.lock` con `uv lock` o eliminar su referencia en el `Dockerfile`.

6.10 Aumentar o externalizar el límite de tokens

El límite de 5.000 tokens es insuficiente. Debería ser configurable por variable de entorno y tener un valor por defecto mucho mayor.

7. Diagnóstico técnico detallado

7.1 Estado del árbol de trabajo

Actualmente HEAD está separado en 0eb79c1 (Initial commit). El working tree contiene README.md, .env (untracked, con credenciales reales) y machine_learning_engineering/__pycache__/. El código real está en la rama feature/issue-1.4-data_read_test, commit 5224689. Esto significa que el código fuente no está disponible en el working tree actual; hay que leerlo desde git.

7.2 Diff entre el original y el actual

El cambio más grande es que agent.py pasó de definir root_agent con un pipeline ADK completo a definir ManagerAgent y SubAgent simples. Los archivos de sub_agents/ y shared_libraries/ no se modificaron en los commits de adaptación, por lo que quedaron obsoletos e inutilizados.

7.3 Verificación del import roto

Se verificó que import executor falla cuando se carga machine_learning_engineering.agent como módulo. La razón es que Python busca un módulo de nivel superior llamado executor, pero executor.py reside dentro del paquete machine_learning_engineering.

7.4 Verificación de que __main__ no ejecuta código

El análisis del bloque __main__ muestra claramente que solo se llaman cuatro delegate_task. No hay llamada a execute_with_retries, ni extracción de código, ni parseo de métricas.

7.5 Comparativa de ejecución

Aspecto	Original 338cad9	Actual 5224689
Consultas a API	Sí, vía ADK/Gemini	Sí, vía OpenAI
Ejecución de código	Sí, automática	Solo manualmente
Parseo de score	Sí	No
Selección de mejores	Sí, greedy	No
Depuración	Bucles anidados	3 reintentos simples
submission.csv	Sí	No
Manejo de ./input	Sí	No

8. Conclusión

El repositorio actual tiene dos capas problemáticas: (1) la capa de conexión a LLM está correctamente adaptada a la API OpenAI-compatible de la universidad —las consultas funcionan; (2) la capa de orquestación y ejecución fue reemplazada por una versión demasiado simplificada que pierde toda la lógica de MLE-STAR, por lo que no se ejecuta ni entrena nada.

Para cumplir tu objetivo actual —ejecutar el modelo original con tu API— lo recomendable es: recuperar la arquitectura original, reemplazar solo el cliente LLM y el runner de ADK, y mantener los prompts, la ejecución de código, el parseo de scores y la gestión de estado originales. Esto te dará un sistema funcional que entrena modelos, compara scores y genera submissions, todo usando la API universitaria.